

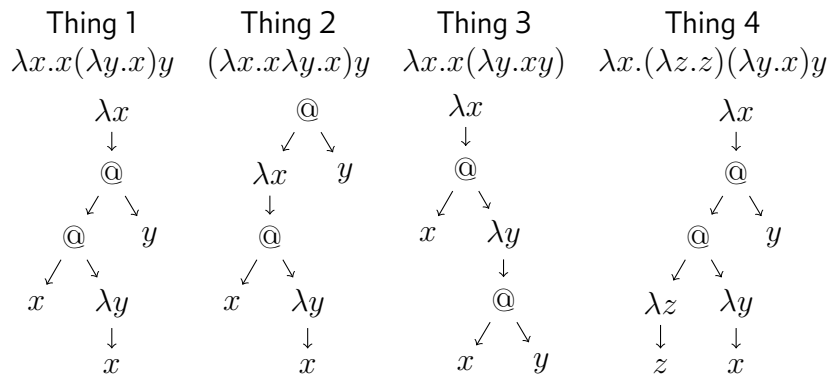
CS 421 --- Lambda Calculus

Name	Netid

Please write your name/netid legibly in dark ink. Hand in one copy per team. Do not staple or mangle the corners.

Reading Syntax and Trees

Consider the following three λ -calculus expressions and their corresponding trees. Note that the text is identical for all three, but we change some parenthesis.



Problem 1) For each of these trees: how many λ abstractions are there? How many application nodes are there?

Problem 2) The standard syntax rules for λ -calculus state that after a λ , the body of the function goes ``as far as it can.'' How is that reflected in the syntax trees?

Problem 3) Thing 1 has an $@$ with another $@$ as its left child. What do you think that means? Which of these would you do first during a reduction?

Problem 4) The y in Thing 3 has a different status than the y in Thing 1 and Thing 2. How does this different status change the meaning of y ?

Problem 5) Here are three new vocab words: *Reducible Expression* (a.k.a. *redex*), *Weak Head Normal Form* (WHNF), and *Normal Form*. Each one of the Things above has one of these three properties. Speculate about Thing is in which form.

Reductions

1. $(\lambda x. \lambda y. f x y) a b \rightarrow_{\beta} (\lambda y. f a y) b \rightarrow_{\beta} f a b$
2. $(\lambda x. \lambda y. f x y) y b \rightarrow_{\beta} (\lambda y. f y y) b \rightarrow_{\beta} f b b$
3. $(\lambda x. \lambda y. f x y) y b \rightarrow_{\alpha} (\lambda x. \lambda y'. f x y') y b \rightarrow_{\beta} (\lambda y'. f y y') b \rightarrow_{\beta} f y b$

Problem 6) Consider these reductions. They have multiple lambdas to start and multiple arguments. In what order do arguments get passed into their parameters?

Problem 7) What is different about the second reduction from the first reduction. Do you think the result is desirable?

Problem 8) The third reduction uses something called $\rightarrow_{\alpha}$. What does it do? Come up with a rule for how and when to use it.

Do some Reductions!

It is essential that you can do lambda calculus reductions. Have each group member take one of the first four and attempt a reduction. If a variable would be α -captured, rename the offending λ . Show your work to each other, and come to a consensus on what the correct answer is.

After that, as a group try the last reduction, but don't spend too much time on it.

Problem 9) $(\lambda x. x) y$

Problem 10) $(\lambda x. x z) (\lambda y. y)$

Problem 11) $(\lambda x. x (\lambda x. y)) (\lambda z. z)$

Problem 12) $(\lambda x. (\lambda y. x)) y (\lambda z. z)$

Problem 13) $(\lambda x. x x) (\lambda x. x x)$

Boolean Lambda Terms

In lambda calculus, we can represent a boolean as a function that takes two parameters. A true function returns the first parameter, and a false function returns the second parameter.

Problem 14) Write the definitions of *True* and *False* as lambda calculus terms.

Problem 15) Write the definitions of *and*, *or*, *not*, and *if*.